Proyecto ARTEMIS — Documento de Diseño Interno

Asistente de Recuperación y Toma de decisiones para Misiones Espaciales
Agencia MASA — Competencia Final MAPLN 2026

---

1. Resumen Ejecutivo

El Proyecto ARTEMIS es la competencia final del curso Modelos Avanzados de PLN. Los estudiantes construyen un sistema de tool calling basado en RAG para la agencia espacial ficticia MASA. El modelo debe decidir qué herramienta invocar (entre 10 tools predefinidas) dado un query de un operador del centro de control, utilizando documentos técnicos recuperados como contexto.

Configuración:

- **Decoder obligatorio:** `meta-llama/Llama-3.2-1B`
- **Encoder obligatorio:** `BAAI/bge-small-en-v1.5`
- **Equipos:** 5 personas
- **Duración:** ~2 semanas
- **GPU:** 24GB VRAM, 128GB RAM
- **Idioma:** Inglés (con nombres OOV en español/quechua)
- **Evaluación:** Exact match en Kaggle (Categorization Accuracy)

---

2. Universo MASA

2.1 La Agencia

- **MASA** — Misión Aeroespacial Sudamericana Avanzada
- Fundada en 2028, sede en Bogotá, Colombia

2.2 Kuntur Station

Estación espacial en órbita baja terrestre con 6 módulos:

Módulo	Función	Notas
Cóndor	Comando y control	Módulo principal
Quetzal	Laboratorio científico	4 tripulantes, 12 workstations
Jaguar	Soporte vital y sistemas críticos	Redundancia clase A
Colibrí	Comunicaciones y navegación	Antenas de largo alcance
Vicuña	Almacenamiento y carga	Puerto de docking
Tucán	Habitacional / dormitorios	6 cabinas individuales

> **Nota sobre nombres:** Los nombres están intencionalmente en español/quechua para estar fuera del vocabulario (OOV) de Llama-3.2-1B. Esto crea un reto sutil de tokenización (Semana 3 del curso).

2.3 Documentos a generar (~60-80)

- Manuales técnicos de cada módulo (specs, rangos operativos, sistemas)
- Protocolos de seguridad: MASA-SEC-001 a MASA-SEC-020
- Procedimientos operativos: MASA-OPS-001 a MASA-OPS-015
- Registros de misiones pasadas (Cóndor-1 a Cóndor-7)
- Perfiles de tripulación (6 miembros)
- Guías de sistemas (ventilación, energía, comunicaciones)
- Normativas y reglamentos internos

---

3. Pipeline End-to-End

```
Query del operador
        ↓
   ENCODER (bge-small-en-v1.5) → genera embedding del query
        ↓
   FAISS → busca documentos similares en la base vectorizada
        ↓
   DECODER (Llama-3.2-1B) recibe: query + contexto RAG + tools
        ↓
   Genera tool call en formato canónico O "no_action"
```

3.1 Ejemplo completo

Query: `"Sensor T-42 in Quetzal shows 78°C. Take appropriate action."`

Contexto RAG recuperado: `"Protocol MASA-SEC-007: temperatures between 45-60°C require HIGH severity alert. Temperatures above 60°C require CRITICAL severity alert and immediate crew evacuation..."`

Output esperado:

```
send_alert(module='quetzal',severity='critical',reason='abnormal_temperature')
```

Por qué funciona: El modelo necesita el contexto RAG para saber que 78°C > 60°C → severity es "critical" (no "high"). Sin buen retrieval, elegirá mal.

---

4. Las 10 Tools

Todas con parámetros enum (valores de un set cerrado). No hay parámetros de texto libre.

4.1 Monitoreo y telemetría

`get_telemetry` — Datos de sensores en tiempo real

- `module`: condor | quetzal | jaguar | colibri | vicuna | tucan
- `metric`: temperature | pressure | oxygen | radiation | humidity | power
- `timeframe_hours`: 1 | 6 | 12 | 24

`get_crew_status` — Estado de la tripulación

- `module`: [6 módulos]
- `info`: health | location | current_activity | schedule

`get_module_status` — Estado general de un módulo

- `module`: [6 módulos]
- `system`: life_support | power | thermal | structural | communications

4.2 Alertas y comunicación

`send_alert` — Alerta a control de misión

- `module`: [6 módulos]
- `severity`: low | medium | high | critical

- `reason`: abnormal_temperature | pressure_drop | oxygen_leak | radiation_spike | system_failure | power_fluctuation | communication_loss | structural_damage

`send_message` — Mensaje a tripulante o equipo

- `recipient`: commander | pilot | specialist_1 | specialist_2 | engineer | medical_officer | all_crew
- `priority`: low | medium | high | urgent

4.3 Operaciones

`schedule_maintenance` — Programar mantenimiento

- `module`: [6 módulos]
- `task`: sensor_repair | filter_replacement | system_calibration | hull_inspection | power_cell_swap | software_update
- `priority`: routine | urgent

`activate_protocol` — Activar protocolo MASA

- `protocol_id`: MASA-SEC-001 ... MASA-SEC-020
- `scope`: module_only | station_wide

`control_system` — Controlar sistema específico

- `module`: [6 módulos]
- `system`: ventilation | heating | lighting | cooling | filtration
- `action`: activate | deactivate | increase | decrease

4.4 Navegación y misión

`calculate_trajectory` — Cálculo orbital

- `maneuver`: orbit_adjustment | docking | debris_avoidance | reentry | station_keeping
- `urgency`: planned | immediate

`request_supply` — Solicitar suministros

- `category`: medical | food | equipment | fuel | spare_parts | scientific
- `urgency`: routine | expedited | emergency

4.5 Acción especial

`no_action` — Cuando ninguna tool aplica a la consulta.

---

5. Formato de Evaluación

5.1 Submission en Kaggle

```csv
ID,answer
1,send_alert(module='quetzal',severity='critical',reason='abnormal_temperature')
2,get_telemetry(module='condor',metric='pressure',timeframe_hours=1)
3,no_action
4,schedule_maintenance(module='jaguar',task='sensor_repair',priority='urgent')
```

5.2 Formato canónico estricto

- Sin espacios después de comas
- Parámetros en el orden definido por la tool
- Strings con comillas simples
- Todo en minúsculas
- Números sin comillas (ej: `timeframe_hours=1` no `timeframe_hours='1'`)
- Post-procesamiento del output permitido

5.3 Métrica

Categorization Accuracy — Exact match del string completo. Si el string generado coincide exactamente con el gold standard → 1, si no → 0.

5.4 Tres capas de evaluación

1. **Tablero público (~60%):** Visible durante la competencia
2. **Tablero privado (~40%):** Se revela al cerrar
3. **Preguntas ocultas (~50-100):** Nosotros corremos su notebook

Score final = promedio de los tres

---

6. Mini-trampas Diseñadas

Trampa 1: Nombres OOV (Tokenización)

- Nombres como Vicuña, Quetzal, Cóndor, Tucán están fuera del vocabulario de Llama
- El tokenizer los parte en subpalabras
- **Impacto:** ~2-5% de accuracy
- **Solución:** Agregar tokens especiales, o simplemente más datos de entrenamiento
- **Conexión curso:** Semana 3 (BPE/WordPiece)

Trampa 2: Severity requiere RAG

- La severity correcta (low/medium/high/critical) solo se determina leyendo el protocolo MASA-SEC
- Ejemplo: 78°C → protocolo dice >60°C = critical, no high
- **Impacto:** Alto. Sin buen retrieval, el modelo elige mal la severity

Trampa 3: Tools ambiguas

- Queries donde varias tools parecen aplicar
- "Temperature rising in Jaguar" → ¿get_telemetry? ¿send_alert? ¿control_system?
- Solo el contexto RAG define la acción correcta

Trampa 4: no_action

- Preguntas donde la respuesta correcta es no hacer nada
- Modelos con fine-tuning agresivo tienden a siempre generar tool calls
- Los equipos deben balancear para que el modelo sepa cuándo abstenerse

Trampa 5: Datos disfrazados para encoder

- "Consultas históricas del centro de control" = pares (query, doc_id)
- Nunca decimos que sirve para fine-tunear el encoder
- Los equipos curiosos descubren que pueden usarlo para contrastive learning
- **Impacto:** +5-15% mejora en retrieval → mejor accuracy final

Trampa 6: Chunking importa

- Documentos con secciones interdependientes

- Si el chunk corta un protocolo a la mitad → contexto incompleto → parámetros incorrectos
- Los equipos con chunking inteligente (overlap, respetar secciones) tienen ventaja

---

7. Técnicas que pueden probar los estudiantes

7.1 Encoder / Retrieval

Técnica	Descripción	Conexión curso
Fine-tuning contrastivo	MNRL loss con sentence-transformers	Semana 5 (BERT)
Hard negative mining	Negativos de módulos similares	Semana 5
Chunking strategies	Tamaño, overlap, metadata	Semana 7 (RAG)
Hybrid search	BM25 + embeddings	Semana 7
Reranking	Cross-encoder para re-ordenar resultados	Semana 7

7.2 Decoder / Generación

Técnica	Descripción	Conexión curso
LoRA / QLoRA	Fine-tuning eficiente	Semana 6
Full fine-tuning	Todos los pesos	Semana 6
Instruction tuning Alpaca	Format system/user/assistant	Semana 7
Data augmentation	LLMs open-source para variaciones	Semana 7
Curriculum learning	Fácil → difícil	General
Tokens especiales	Agregar tokens OOV al vocab	Semana 3
LR scheduling	Warmup, cosine decay	General

7.3 Pipeline / Post-procesamiento

Técnica	Descripción	Conexión curso
Prompt engineering	Estructura del prompt	Semana 7

Técnica	Descripción	Conexión curso
Generación controlada	Temperature baja, constrained	Semana 6
Post-procesamiento	Regex para normalizar formato	Práctica
Few-shot en prompt	Ejemplos en system prompt	Semana 7

8. Entregables

8.1 Lo que entregamos nosotros

```
📁 proyecto_artemis/
├── 📁 base_conocimiento/
│   └── documentos_masa.json              ← ~60-80 docs técnicos
├── 📁 datos_entrenamiento/
│   ├── train_tool_calls.json             ← query → tool call correcto
│   ├── consultas_centro_control.json     ← pares (query, doc_id) — pista encoder
│   └── bitacora_incidentes.json          ← escenarios completos
├── 📁 evaluacion/
│   ├── test_queries.csv                  ← queries para Kaggle (~1000)
│   └── sample_submission.csv
├── tools_definition.json                 ← definición de las 10 tools
└── README.md
```

8.2 Lo que entregan los estudiantes

En Kaggle:

- CSV con `ID,answer` (tool call como string)
- Mínimo 5 submissions

En Coursera:

- Notebook de entrenamiento (encoder + decoder)
- Notebook de inferencia (pipeline completo: query → tool call)
- Checkpoint del decoder (Llama fine-tuneado)
- Checkpoint del encoder (si fine-tunearon)
- Índice FAISS / vectores pre-computados
- Corpus de entrenamiento usado

- `requirements.txt` + `instrucciones.txt`

8.3 Lo que nosotros generamos (backlog)

1. ✅ Universo MASA completo (~60-80 documentos coherentes)
2. ✅ 10 tools con enums completos (JSON)
3. ✅ Dataset de entrenamiento (~500-1000 ejemplos)
4. ✅ Dataset de test (~1000 preguntas Kaggle + ~100 ocultas)
5. ✅ Gold standard verificado manualmente
6. ✅ Baseline (Llama sin fine-tuning)
7. ✅ Datos disfrazados (consultas históricas, bitácora)
8. ✅ Enunciado PDF + rúbrica
9. ✅ Montar competencia en Kaggle

---

9. Estimación de solución ganadora

Pipeline óptimo

1. **Chunking:** 256-512 tokens, overlap 50, respetar secciones, metadata
2. **Encoder fine-tuning:** MNRL loss con consultas históricas. Hard negatives. 3-5 epochs, lr=2e-5
3. **Decoder QLoRA:** Rank=16, alpha=32. Data augmentation 3x con LLM open-source. 5-10 epochs, lr=1e-4
4. **Retrieval:** Hybrid BM25+embeddings → top-10 → reranking → top-3
5. **Post-procesamiento:** Regex normalización, validar tool/params, fallback no_action
6. **Tokens especiales:** Quetzal, Vicuña, Cóndor, Tucán en tokenizer

Accuracy estimada: 70-85%

---

10. Rúbrica

Ítem	Porcentaje	Detalle
Participación	20%	Mínimo 5 envíos a Kaggle
Superar baseline	40%	Promedio público+privado+oculto > modelo base. Si no → 0.0

Ítem	Porcentaje	Detalle
Percentil	40%	<P25→0% · ≥P25→50% · ≥P50→75% · ≥P75→100%

Restricciones

✅ **Permitido:**

- Fine-tuning del encoder (opcional)
- Modificar tokenizer de Llama
- Data augmentation con LLMs open-source
- Reranking y reprompting
- Hybrid search (BM25 + embeddings)
- Post-procesamiento del output
- Few-shot examples en prompt
- Curriculum learning, LR scheduling

🚫 **Prohibido:**

- Modelos pagos (OpenAI, Claude, Cohere, Gemini APIs)
- Datos externos no entregados por el equipo docente
- Usar información del test set para entrenar
- Clasificación con otra red neuronal en vez del LLM
- Solo prompt engineering sin fine-tuning del decoder
- Usar decoder diferente a Llama-3.2-1B
- Usar encoder diferente a bge-small-en-v1.5
- Entrenamiento fuera de PyTorch

---

11. Tres opciones de competencia

Opción 1 — Solo fine-tuning del decoder (contexto dado)

- Nosotros entregamos el contexto RAG ya recuperado
- Ellos solo fine-tunean Llama
- **Pro:** Enfocado, más tiempo para decoder, evaluación justa
- **Con:** No construyen RAG, menos diferenciación, no aplica Sem 5/7
- **Timeline:** 1 sem fine-tuning + 1 sem optimización

★ Opción 2 — Pipeline completo: RAG + decoder (RECOMENDADA)

- Construyen todo: chunking → encoder → FAISS → decoder → tool call
- Fine-tuning encoder opcional (con pistas disfrazadas)
- **Pro:** Completo, realista, múltiples ejes, aplica todo el curso
- **Con:** Complejo para 2 semanas, más puntos de falla
- **Timeline:** Sem 1: chunking+RAG+primeros exp · Sem 2: fine-tuning+optimización

Opción 3 — Solo decoder con memorización (sin RAG)

- Sin retrieval. El modelo memoriza la info durante fine-tuning
- Continual pre-training + instruction tuning
- **Pro:** Técnicamente interesante, más simple
- **Con:** Similar a competencia anterior (estatutos), sin RAG, memorización frágil
- **Timeline:** 1 sem pre-training + instruction tuning · 1 sem optimización

---

12. Timeline General (Opción 2)

Momento	Actividad
Semana 7 — Anuncio	Publicación enunciado, entrega materiales, apertura Kaggle
Semana 7-8 — Construcción	Chunking, FAISS, primeros fine-tuning decoder, primeros submissions
Semana 8 — Optimización	Mejorar retrieval, data augmentation, técnicas avanzadas, submissions frecuentes
Fin Semana 8 — Cierre	Cierre Kaggle, entrega Coursera, evaluación preguntas ocultas

---